

Playwright Framework Review — Constructive Feedback

Project: Ezra Customer Booking UI Automation
Reviewed: 2026-04-05
Framework: Playwright (v1.58.2) + JavaScript (ES Modules)

Executive Summary

The framework has a solid foundation — clean Page Object Model, environment-driven config, and CI/CD with GitHub Actions. However, it underutilizes Playwright's capabilities significantly and has gaps in reliability, security, maintainability, and test coverage that will cause pain as the suite grows.

Below is an honest, categorized breakdown.

1. Architecture & Project Structure

What's Good

- Clear POM separation (`page/`, `tests/`, `util/`, `config/`)
- Environment-driven config via `dotenv` — easy to switch between stage/dev
- ES module usage (`"type": "module"`)

What Needs Improvement

Issue	Detail	Recommendation
No base page class	Every page object constructs its own <code>this.page</code> with no shared behavior	Create a <code>BasePage</code> class with common methods: <code>waitForPageLoad()</code> , <code>screenshot()</code> , <code>getTitle()</code> , navigation helpers. All pages extend it.
Flat page directory	All page objects sit in <code>page/</code> with no grouping	As pages grow, organize by feature: <code>page/auth/</code> , <code>page/booking/</code> , <code>page/payment/</code>
No fixtures directory	Test data is scattered across <code>.env</code> files, hardcoded values, and inline generators	Create <code>fixtures/</code> or <code>data/</code> directory with structured test data (JSON/JS files)
Utils class is dead code	<code>util/utils.js</code> has <code>generateRobustEmail()</code> but <code>loginPage.js</code> has its own duplicate	Remove duplication — pick one location and use it everywhere
No TypeScript	<code>@types/node</code> is installed but everything is <code>.js</code>	Migrate to TypeScript. You get autocomplete, type safety, and better refactoring support. Playwright has first-class TS support — zero config needed.

2. Playwright Configuration (`playwright.config.js`)

Current State

```
export default defineConfig({
  reporter: [['html', { open: 'never' }]],
  timeout: 180 * 1000,
  use: {
    baseURL: process.env.BASE_URL,
    headless: false
  }
});
```

Problems & Recommendations

Issue	Impact	Fix
No projects defined	Tests only run on one browser	Add multi-browser projects (Chromium, Firefox, WebKit). This is a core Playwright advantage over Selenium.
headless: false hardcoded	CI runs will open a browser window (or fail)	Use headless: true for CI. Control via env variable: headless: !!process.env.CI
No retries configured	Flaky tests fail immediately	Add retries: process.env.CI ? 2 : 0 — retry in CI, not locally
180s timeout is excessive	Slow failures, slow feedback loop	Reduce to 60s. If a test needs 3 minutes, it's doing too much.
No expect timeout	Default 5s may not match your app's loading patterns	Set expect: { timeout: 10000 } explicitly
No testDir	Relies on default	Explicitly set testDir: './tests'
No outputDir	Test artifacts (screenshots, traces) go to default location	Set outputDir: './test-results'
No trace configuration	No traces captured on failure	Add trace: 'on-first-retry' — invaluable for debugging CI failures
No screenshot on failure	Failed tests leave no visual evidence	Add screenshot: 'only-on-failure'
No video recording	No way to replay failures	Add video: 'retain-on-failure' for CI debugging

Issue	Impact	Fix
No <code>globalSetup/globalTeardown</code>	Login repeats in every test	Use <code>globalSetup</code> for auth state, save <code>storageState</code> to reuse across tests
Single reporter	Only HTML output	Add multiple: <code>[['html', { outputFile: 'results.json' }], ['list', { outputFile: 'results.json' }]]</code>

Recommended Config

```
export default defineConfig({
  testDir: './tests',
  outputDir: './test-results',
  timeout: 60_000,
  retries: process.env.CI ? 2 : 0,
  workers: process.env.CI ? 1 : undefined,
  reporter: [
    ['html', { open: 'never' }],
    ['list'],
  ],
  use: {
    baseUrl: process.env.BASE_URL,
    headless: !!process.env.CI,
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
  },
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
    { name: 'webkit', use: { ...devices['Desktop Safari'] } },
  ],
});
```

3. Page Object Model Implementation

What's Good

- Each page has its own class with locators and actions
- Use of `getByRole()` for some locators (accessibility-friendly)
- Cookie acceptance handled conditionally

What Needs Improvement

3a. Locator Strategy Issues

Current	Problem	Better Approach
---------	---------	-----------------

Current	Problem	Better Approach
<code>#email, #password</code>	CSS ID selectors are fragile	Use <code>page.getByLabel('Email')</code> , <code>page.getByPlaceholder('Email')</code> — Playwright's recommended locators
<code>li[data-testid="FB30-encounter-card"]</code>	Good use of <code>data-testid</code> , but chained with <code>.filter({ hasText: 'MRI Scan' })</code> would be cleaner	<code>page.getByTestId('FB30-encounter-card').filter({ hasText: 'MRI Scan' })</code>
XPath locators throughout <code>scheduleScanPage.js</code>	XPath is brittle, hard to read, and not recommended by Playwright	Replace with Playwright locators: <code>getByRole()</code> , <code>getByText()</code> , <code>getByTestId()</code> , <code>locator().filter()</code>
<code>.multiselect__select</code>	Implementation-detail CSS class	Use <code>getByRole('combobox')</code> or <code>getByLabel('Gender')</code>

Rule of thumb: Playwright's locator priority should be: `getByRole` > `getByLabel` > `getByPlaceholder` > `getByText` > `getByTestId` > CSS > XPath (last resort).

3b. Wait Strategy Issues

Current Code	Problem	Fix
<code>await this.page.waitForTimeout(2000)</code>	Hard-coded waits — the #1 cause of flaky tests	Use <code>await expect(locator).toBeVisible()</code> or <code>locator.waitFor()</code>
<code>await this.page.waitForTimeout(5000)</code>	5 seconds of wasted time on every run	Wait for a specific condition: network idle, element visible, URL change
Multiple <code>waitFor({ state: 'visible', timeout: 10000 })</code>	Inconsistent timeout values scattered everywhere	Centralize timeouts in config or base page

Playwright auto-waits on actions. When you call `locator.click()`, it already waits for the element to be visible and stable. Explicit waits should only be needed for assertions or non-standard conditions.

3c. Page Object Design Issues

Issue	Detail
Methods accept <code>page</code> as parameter	<code>selectState(page, stateName)</code> — but <code>page</code> is already <code>this.page</code> from the constructor. Remove the parameter.
Business logic in page objects	<code>generateRobustEmail()</code> , <code>generateUSPhone()</code> , <code>generateRandomName()</code> belong in a data utility, not a page object

Issue	Detail
No return types or JSDoc	Methods return <code>undefined</code> , <code>true</code> , or <code>false</code> inconsistently with no documentation
Login method returns boolean	<code>login()</code> returns <code>true/false</code> — tests should use assertions, not return values. Let the method throw on failure.
<code>createNewUser()</code> does too much	Generates data AND fills form AND submits — break into smaller methods

4. Test Design & Quality

Current Tests

- `login.spec.js` — 3 tests (login, negative login, create user)
- `booking.spec.js` — 1 test (MRI scan scheduling end-to-end)

Issues

Issue	Detail	Fix
Suite naming mismatch	<code>booking.spec.js</code> has <code>describe('Login Tests')</code>	Name the suite correctly: <code>describe('Booking Tests')</code>
Insufficient assertions	Booking test only asserts one final element	Add intermediate assertions: plan selected, location selected, date confirmed, payment processed
No negative/edge case tests	Only 1 negative test (invalid login)	Add: empty fields, SQL injection attempts, XSS payloads, boundary values, session timeout
No independent test isolation	Booking test depends on login in <code>beforeEach</code>	Use Playwright's <code>storageState</code> to skip UI login after first auth
<code>test.setTimeout(60000)</code> inside test	Timeout should be in config, not scattered in tests	Move to config or use <code>test.slow()</code> for known slow tests
No <code>test.describe.configure</code>	Tests may run in parallel unsafely	Use <code>test.describe.configure({ mode: 'serial' })</code> if tests depend on order
No test tagging	Can't run smoke vs. regression vs. full suites	Use <code>test.describe('', { tag: '@smoke' })</code> and filter with <code>--grep</code>
No API-level setup	UI login in every test suite	Use <code>request</code> fixture for API-based test setup (create users, seed data via API)

Issue	Detail	Fix
Hardcoded test data	Date 10, month 4, state California, location North Irvine	Parameterize with test data files or use <code>test.describe</code> with data sets

5. Security Issues (Critical)

Issue	Severity	Fix
Credentials in <code>.env</code> file committed to repo	HIGH	<code>stage.env</code> contains real email/password. Add <code>config/env/*.env</code> to <code>.gitignore</code> . Use CI secrets for pipeline.
Stripe test card hardcoded in source	MEDIUM	Move to env variables or test data files (even test cards shouldn't be in source code for hygiene)
Password visible in plain text	HIGH	<code>Peacock2025!</code> and <code>DoTEST2025!</code> in source. Use a secrets manager or environment variables from CI.
<code>.env</code> files not in <code>.gitignore</code>	HIGH	These files are tracked by git. Fix immediately.

6. CI/CD Pipeline (`playwright.yml`)

What's Good

- GitHub Actions setup is functional
- Artifact upload for reports (30-day retention)
- Uses `npm ci` for deterministic installs

What Needs Improvement

Issue	Fix
No environment secrets	Use GitHub Secrets for <code>BASE_URL</code> , <code>EMAIL</code> , <code>PASSWORD</code> instead of <code>.env</code> files
No parallel sharding	Add <code>shardIndex</code> and <code>shardTotal</code> for faster CI runs
No caching	Add Node modules caching to speed up installs
Triggers only on main/master	Add trigger for <code>dev</code> branch (your default branch)
<code>headless: false</code> in config	CI will fail or run slowly. Must be <code>headless: true</code> in CI
No Slack/Teams notification	No alerts on failure — add notification step

Issue	Fix
No test result summary	Use <code>dorny/test-reporter</code> or similar for PR comments with results
No scheduled runs	Add <code>schedule</code> trigger for nightly regression runs

7. Underutilized Playwright Features

These are Playwright capabilities you're not using at all:

Feature	What It Does	How to Use
<code>storageState</code>	Save/reuse auth state (cookies, localStorage)	Login once in <code>globalSetup</code> , reuse in all tests — massive speed improvement
<code>test.step()</code>	Break tests into named steps in reports	<code>await test.step('Fill payment form', async () => { ... })</code>
<code>expect</code> soft assertions	Continue test after assertion failure	<code>expect.soft(locator).toBeVisible()</code> — collect all failures
API testing (<code>request</code>)	Built-in HTTP client	Replace Postman collections with Playwright API tests. Keep everything in one framework.
<code>test.fixme()/test.skip()</code>	Mark tests as known issues	Better than commenting out tests
Network interception	Mock API responses	<code>page.route('**/api/**', route => route.fulfill({ body: '...' })))</code> — test error states without real failures
Visual comparisons	Screenshot diffing	<code>expect(page).toHaveScreenshot()</code> — catch UI regressions automatically
Accessibility testing	a11y audit	<code>expect(page).toPassAccessibilityChecks()</code> or integrate <code>@axe-core/playwright</code>
<code>devices</code> presets	Mobile/tablet testing	<code>use: { ...devices['iPhone 13'] }</code> — test responsive layouts
Custom fixtures	DI for test dependencies	Create typed fixtures for page objects instead of manual instantiation
<code>testInfo</code> annotations	Rich test metadata	Add links to tickets, owners, severity in reports

Feature	What It Does	How to Use
Parallel workers	Run tests concurrently	Configure <code>workers</code> in config — tests should be independent enough for this
<code>toPass()</code> polling	Retry assertion block	<code>await expect(async () => { ... }).toPass()</code> for eventually-consistent checks

8. Code Quality & Maintainability

Issue	Recommendation
No linter configured	Add ESLint with <code>@typescript-eslint</code> rules
No formatter	Add Prettier with consistent config
No pre-commit hooks	Add <code>husky</code> + <code>lint-staged</code> to enforce quality on commit
No JSDoc or type annotations	At minimum, add JSDoc for public methods. Ideally, migrate to TypeScript.
Console logs in page objects	Remove <code>console.log</code> statements — use Playwright's built-in <code>test.info()</code> for debugging
Magic numbers everywhere	<code>2000</code> , <code>5000</code> , <code>10000</code> — extract into named constants
No error screenshots	When a step fails, capture a screenshot. Playwright can do this automatically with <code>screenshot: 'only-on-failure'</code> .

9. Missing Test Types

Test Type	Status	Action
Smoke tests	Missing	Tag critical-path tests with <code>@smoke</code> for quick validation
API tests	Postman only	Migrate to Playwright <code>request</code> — single framework, single report
Visual regression	Missing	Add <code>toHaveScreenshot()</code> comparisons
Accessibility (a11y)	Missing	Integrate <code>@axe-core/playwright</code>
Performance checks	Missing	Use <code>page.evaluate(() => performance.getEntries())</code> for basic metrics
Mobile/responsive	Missing	Add mobile device projects in config
Cross-browser	Missing	Add Firefox and WebKit projects

Test Type	Status	Action
Data-driven/parameterized	Missing	Use arrays or JSON files with loops for test data variation

10. Priority Action Items

Immediate (Do Now)

1. **Fix security issues** — Remove credentials from tracked files, add `.env` to `.gitignore`, use CI secrets
2. **Fix headless for CI** — Pipeline is likely broken or wasteful
3. **Add retries and trace: 'on-first-retry'** — Instant reliability improvement
4. **Fix suite name in `booking.spec.js`** — 'Login Tests' -> 'Booking Tests'
5. **Remove `waitForTimeout()` calls** — Replace with proper waits

Short-Term (This Sprint)

6. **Implement `storageState` auth** — Skip UI login after first test, save minutes per run
7. **Add multi-browser projects** — Chromium, Firefox, WebKit
8. **Consolidate duplicate code** — Remove duplicate `generateRobustEmail`, eliminate dead `Utils` class
9. **Remove `page` parameter from methods that already have `this.page`**
10. **Replace XPath locators** with Playwright-native locators

Medium-Term (Next 2-4 Weeks)

11. **Migrate to TypeScript**
12. **Create `BasePage` class** with shared functionality
13. **Add visual regression tests** with `toHaveScreenshot()`
14. **Migrate Postman API tests** to Playwright `request`
15. **Add custom Playwright fixtures** for page object injection
16. **Set up ESLint + Prettier + Husky**

Long-Term (Next Quarter)

17. **Add accessibility testing** with axe-core
18. **Add mobile device testing** via Playwright device emulation
19. **Implement test data management** with factories/builders
20. **Add performance baseline checks**
21. **Implement parallel test sharding in CI**

Summary Scorecard

Category	Score	Notes
Project Structure	6/10	Good foundation, needs TypeScript and better organization

Category	Score	Notes
Playwright Config	3/10	Minimal — missing retries, traces, multi-browser, proper headless
Page Objects	5/10	POM exists but has dead code, XPath, hardcoded waits, design issues
Test Quality	4/10	Very few tests, minimal assertions, no parameterization
Security	2/10	Credentials in repo, no secrets management
CI/CD	5/10	Functional but missing caching, secrets, notifications
Playwright Utilization	3/10	Using maybe 20% of what Playwright offers
Overall	4/10	Solid starting point, significant room for improvement

Review conducted by Automation Architect. This is meant to be constructive — the foundation is there, and addressing these items systematically will turn this into a production-grade framework.